

COMSATS University Islamabad, Lahore Campus
CSC241 – Object Oriented Programming (Java): Theory Assignment 1

Object Oriented Analysis of a Smart Campus Allocation System

Submitted by: Roshan Ahmed | SP26-BAI-043 | BS Artificial Intelligence, Section A

Submitted to: Dr. Muhammad Shahid Bhatti

Task 1 — Stakeholder and Requirement Analysis

Stakeholders and needs:

Stakeholder	Primary need
Students	Fair access to rooms and scarce equipment.
Faculty	Guaranteed rooms/equipment for scheduled teaching; priority for urgent academic needs.
Administration	High utilization of rooms and equipment, with reliable reports.
Maintenance staff	Ability to block an unsafe room or faulty unit immediately.

Genuine conflicts:

Conflict	Assumption needed before design
Fair access vs. faculty priority [RL]	Timetabled teaching cannot be displaced; urgent requests may only displace unconfirmed student bookings. Fairness capped at 6 active student hours/week, tracked separately for rooms and equipment.
Privacy vs. traceability	Every allocation is logged in an append-only audit trail; only users with ViewAudit permission see identities. Others see only free/busy status.
Utilization vs. safety	Safety always wins — a block or closure takes effect immediately. The system then tries to relocate or substitute before cancelling.

Facts vs. assumptions:

Fact given	Assumption introduced
Four activity types, each with different capacity/equipment/approval rules.	Classes and events need approval; individual/group requests auto-approve if rules are met (PendingApproval state).
Maintenance blocks unsafe rooms immediately; audit trail required; privacy expected.	Recurring reservations run one semester and are stored individually so each can be disrupted separately.
Emergencies may cancel/relocate; affected users must be notified.	Relocation priority: teaching, then accessibility needs, then events, then others. Time slot is kept unless the user consents to a change.

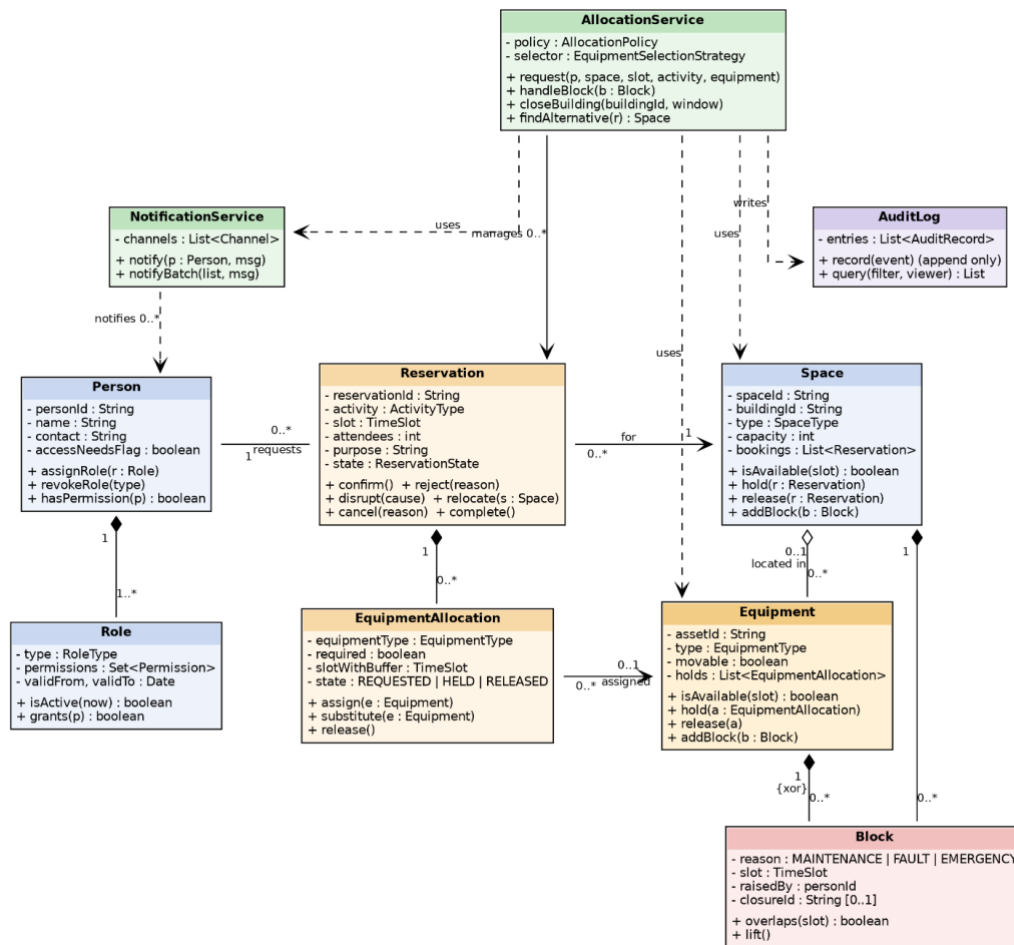
Fact given	Assumption introduced
Equipment is movable and scarce [RL].	Room and equipment are allocated separately; a booking confirms only when both are held. Movable units need a 15-min transfer buffer; fixed equipment stays with its room.

Task 2 — Object Oriented Conceptual Model

Proposed classes:

Class	Responsibility	Key attributes / operations
Person	Represents a user; answers permission checks via its roles.	personId, name, accessNeedsFlag; assignRole(), hasPermission()
Role	One role held for a period, with its permissions.	type, permissions, validFrom/To; isActive(), grants()
Space	A bookable room; owns its schedule, rejects overlaps.	buildingId, capacity, bookings; isAvailable(), hold(), addBlock()
Reservation	One booking; controls its own life cycle.	activity, slot, state; confirm(), disrupt(), relocate(), cancel()
Equipment [RL]	One physical unit; owns its schedule, rejects overlaps.	assetId, movable, holds; isAvailable(), hold(), addBlock()
EquipmentAllocation [RL]	Links a reservation to one equipment unit.	slotWithBuffer, state; assign(), substitute(), release()
Block	A period a space/unit is unusable (maintenance, fault, emergency).	reason, slot, closureId; overlaps(), lift()
AuditLog	Append-only record; filters what each viewer may see.	entries; record(), query(filter, viewer)

UML class diagram:



Responsibility separation: no class runs the whole system. Space and Equipment each guard their own schedule since each owns its bookings/holds. Reservation alone governs its legal state changes. Person/Role answer permission questions; AuditLog alone controls who can read identities. AllocationService only coordinates — it cannot write into a schedule directly. [RL] Because equipment has its own allocation object, a faulty projector is fixed by substituting one EquipmentAllocation, not by cancelling the whole reservation.

Task 3 — Abstraction, Encapsulation and Invariants

Class	What it abstracts
Reservation	An agreement to use a room and its needed equipment at a time — keys and paperwork are ignored; only what matters for allocation, audit and relocation is kept.
Space	A physical room reduced to capacity, type and calendar — décor and exact dimensions are ignored.
Equipment [RL]	A unit reduced to type, mobility, location and busy times — serial numbers and purchase data are ignored.

Class	Hidden information	Controlled via
Reservation	state, purpose, attendee list, equipment allocations	confirm(), disrupt(), relocate(), cancel() — no public setState()

Class	Hidden information	Controlled via
Space	bookings, blocks list	isAvailable(), hold(), release(), addBlock() — callers never get the raw list
Equipment [RL]	holds, blocks, current location	isAvailable(), hold(), moveTo() — only moveTo() changes location, and it refuses fixed units

Invariants:

Invariant	How encapsulation protects it
No two confirmed reservations for the same space overlap.	Only Space.hold() writes to bookings; it checks overlap and adds atomically.
No equipment unit is held by two overlapping allocations, including the 15-min buffer [RL].	Only Equipment.hold() writes to holds, using a buffer computed inside EquipmentAllocation — not supplied by callers.
A reservation is Confirmed only while its room and every required allocation are held [RL].	confirm() checks this; if a unit is later released, the allocation notifies its reservation, moving it to Disrupted.
Nothing is confirmed for a space/unit during an overlapping block.	hold() refuses blocked slots; addBlock() triggers disrupt()/substitute() on overlapping bookings.

Task 4 — Alternative Designs and Decision

Option A — Inheritance: an abstract User superclass with subclasses Student, Faculty, Admin, MaintenanceStaff, SecurityStaff. A student who is also a TA needs a further StudentTA subclass.

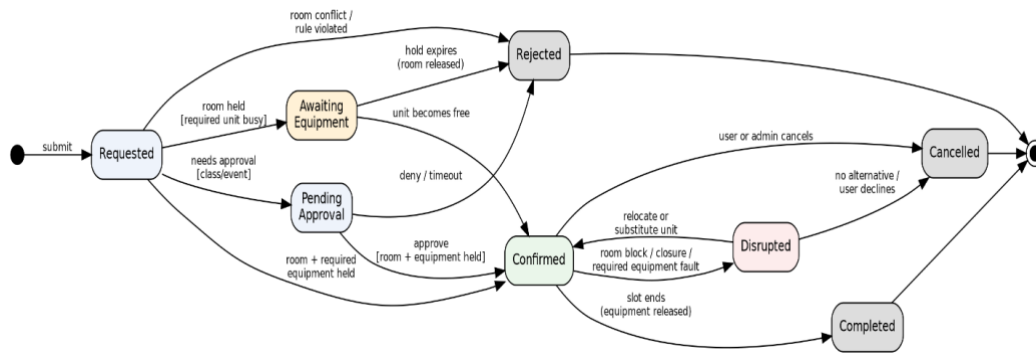
Option B — Composition (role objects): a single Person class holds a collection of Role objects, each with its own permissions and validity dates.

Criterion	A: Inheritance	B: Composition
Duplicate behaviour	Combination subclasses repeat permission logic (Java has no multiple inheritance).	Permission logic lives once in Role; Person just combines results.
Changing permissions [RL]	Letting TAs book scarce equipment means editing subclass code and redeploying.	Add the permission to a TA role with an expiry date — no code change.
Coupling	Allocation code tends to use instanceof checks, tying it to the hierarchy.	Allocation code only calls hasPermission(p) — low coupling.

Decision: Option B. Roles and permissions change during the semester and one person may hold several — inheritance can't represent that without recreating objects. Scarce-equipment access is granted and withdrawn often (e.g., a TA only while a lab runs); as a role object this is one assignRole() call that lapses automatically at validTo.

Task 5 — Behaviour and Edge Cases

Reservation life cycle:



Edge case	Model response	Enforced by
Two simultaneous requests for the same space/last unit [RL]	Space.hold() checks and adds atomically, so the first commit wins; the loser gets a conflict and findAlternative(). For the last equipment unit, Equipment.hold() decides the same way — the loser proceeds without it (optional) or its room hold rolls back (required). Locks are taken room-first, then equipment in assetId order, to avoid deadlock.	Space and Equipment each own their schedule; AllocationService only sets priority, it can't bypass the checks.
Maintenance block after confirmation	Space.addBlock() takes effect immediately; handleBlock() marks overlapping reservations Disrupted and searches for an equivalent room. [RL] Movable units follow the booking; fixed equipment requires the new room already have it. A faulty unit alone triggers substitute() without touching the room booking.	Space/Equipment enforce the block; Reservation enforces legal transitions; AllocationService coordinates relocation.
Emergency cancellation or relocation	closeBuilding() adds one EMERGENCY block (shared closureId) to every space and unit in the building. Affected reservations are relocated using resources outside the closed building, or cancelled if none remain; one notifyBatch() informs users.	AllocationService owns relocation policy; Block records scope; AuditLog keeps records separate from the decision.

Task 6 — Independent Exploration: the Strategy Pattern

The Strategy pattern puts each version of an algorithm in its own class behind a shared interface, so the object using it can switch versions at run time without changing its own code. The client depends only on the interface, not on any implementation, so it stays stable as rules change.

Applied here [RL]: when several units of a requested type are free, AllocationService doesn't choose itself — it calls selector.select(candidates, reservation) on an EquipmentSelectionStrategy field. NearestUnitStrategy picks the closest unit to cut moving time; BalancedWearStrategy picks the least-used unit so equipment ages evenly; ReserveForTeachingStrategy refuses to give students the last free unit during a teaching week. Administration can swap strategies weekly with no change to AllocationService, and once the university defines fair access, a new strategy class is added instead of extending an if-else chain. Each strategy is also independently testable.

References:

[1] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1995, pp. 315–323.

[2] E. Freeman and E. Robson, Head First Design Patterns, 2nd ed. Sebastopol, CA: O'Reilly Media, 2020, ch. 1.

AI disclosure: [state here how, if at all, a generative AI tool was used, per the submission requirements.]